

AURIX API Contract: Scan Results JSON Schema

Version: 2.0 (Post-AWS Deployment)

Producer: Divyansh (AWS EC2 LangGraph Engine)

Consumers: Bhavya (Render Backend API), Bhumika (React Web Dashboard), Divyanshi (VS Code Extension)

Overview

This document defines the strict JSON payload that the AI Engine will output upon completing a repository scan.

- **Bhavya:** Use this schema to design the Supabase database tables (**Scans** and **Vulnerabilities**).
 - **Bhumika & Divyanshi:** Use this schema as your mock data to build your user interfaces (Kanban boards, VS Code squiggly lines) before Bhavya finishes the backend.
-

The Payload Structure

The AWS engine will send a single JSON object via a Webhook POST request containing top-level metadata and an array of findings.

Example JSON Payload

```
{
  "scan_id": "829fcec8-6b82-4f94-a794-35ebf4e222cb",
  "url": "https://github.com/stamparm/DSVW",
  "timestamp": "2026-07-19T14:42:00.000Z",
  "summary": {
    "total_findings": 12,
    "neutralized_count": 8,
    "scan_engine": "Project AURIX LangGraph v1"
  },
  "findings": [
```

```
// ... Array of Finding Objects

]

}
```

The **Finding** Object Schema

Each object inside the **findings** array strictly adheres to the following structure.

1. Base Properties (Always Present)

These fields are generated during Phase 1 (The initial static scan by Opengrep, Trivy, Gitleaks, Hadolint).

- **id (String/UUID)**: Unique identifier for the finding.
- **rule_id (String)**: The internal rule that triggered (e.g., `python.lang.security.deserialization`).
- **tool (String)**: The scanner that found it (e.g., `opengrep`, `trivy`, `hadolint`).
- **category (String)**: Vulnerability category (`sast`, `infra`, `secret`).
- **title (String)**: Short human-readable title (e.g., `SQL Injection`).
- **description (String)**: Full explanation of the vulnerability.
- **severity (String)**: Enum [`'LOW'`, `'MEDIUM'`, `'HIGH'`, `'CRITICAL'`].
- **cvss (Float)**: 0.0 to 10.0 score.
- **file (String)**: Relative path to the vulnerable file (e.g., `/src/dsvw.py`).
- **line (Integer)**: The line number where the issue exists.
- **evidence (String)**: The raw snippet of code that triggered the alert.
- **fix (String)**: Static tool recommendation (often vague like "Consult documentation.").
- **references (Array of Strings)**: List of URLs pointing to CVEs or OWASP documentation.

2. Verification Properties (Present if processed by AI)

These fields are appended during Phase 2 (LangGraph Triage and Execution).

- **verified (Boolean)**:
 - `true` = The AI confirmed it is a real threat.
 - `false` = The AI discarded it as a false positive or low-confidence noise.

3. Agentic Artifacts (Optional - Only present if **verified: true**)

These fields are the high-value outputs generated by the LangGraph Agents.

- **wargame_status (String):**
 - "PoC Failed" = AI tried to exploit it but couldn't verify the threat.
 - "Exploit Confirmed" = Red Agent successfully exploited it, but Blue Agent couldn't patch it.
 - "Neutralized" = Perfect success. Exploit confirmed AND patch written.
 - **ai_reasoning (String):** The LLM's explanation of why this bug is real and how the attack path works.
 - **poc_script (String):** Raw Python/Bash code of the exploit script the AI used.
 - **patch_code (String):** Raw code representing the Blue Agent's patch file (intended to be injected by the VS Code extension or GitHub PR).
-

Action Items for the Team

- **Bhumika (Web UI):** Copy the full schema and start building the "Vulnerability Kanban Board" and "Contextual AI Chat" using the **ai_reasoning** field.
- **Divyanshi (VS Code):** Use the **file**, **line**, and **patch_code** fields to build your **DiagnosticsAPI** squiggly lines and your **CodeActionProvider** 1-click Quick Fixes.
- **Bhavya (Backend):** Map these fields directly into your Supabase database schema so you can cleanly parse the Webhook payload.